

Why Runes

A Ruby agent harness with a fabric, an identity, a memory and a face

Runes 0.3.0 | 11 September 2026 | github.com/runes-harness/runes

A Ruby agent harness with a fabric, an identity, a memory, and a face. Fun to build, and quietly one of the more consequential things you could be building right now.

TL;DR

- **Runes runs fleets of LLM agents** over a swappable message fabric (in-process <-> MQTT 3.1.1 <-> MQTT 5), with A2A for discovery and MCP for tools.
- **Every workflow verb is a plugin.** `cmd`, `ruby`, `chat`, `agent`, `map`, `repeat`, `call` - seven runes, between 68 and 728 lines each, all replaceable, and a new one takes about thirty lines.
- **It runs Shopify Roast workflows** unmodified.** The shipped example is Roast's README example, byte for byte, and the test suite reads that exact file.
- **It treats the boring parts as the product:** capability guard, signed envelopes, a durable journal, verify-and-resume, and a Rails observatory that watches the whole fleet.
- **Execution is deduped, not just distributed.** MQTT 5 shared subscriptions hand each prompt to one agent; `Runes::RequestLedger` makes a redelivery or a retry a no-op that still gets an answer - and a refusal is a published event the observatory can count, not a line in a log.
- **14 745 lines of lib, 11 187 lines of tests, 600 tests, zero network in the suite.** Ruby 4.0.4, Rails 8.1.3.1, SQLite, mosquito on localhost.

60 seconds: what it actually is

A workflow is a plain Ruby file that is `instance_eval'd` - no DSL gem, no YAML, no graph builder:

```
execute do
  cmd(:recent_changes) { "git diff --name-only HEAD~5..HEAD" }

  agent(:review) do
    files = cmd!(:recent_changes).lines
    "Review these for security, performance and maintainability:\n#{files.join("\n")}"
  end

  chat(:summary) do
    "Summarize this for non-technical stakeholders:\n\n#{agent!(:review).response}"
  end
end
```

```
bin/runes-workflow execute examples/analyze_codebase.rb
```

`cmd!(:recent_changes).lines` inside the agent block is a method call, not a variable - inside a step, `self` is the step's input context. That single design decision is why the syntax stays readable as workflows grow, and it is Roast's, not ours.

Underneath, the same file drives a fleet over MQTT, where agents are discoverable, addressable, and auditable:

```
runes/agents/<id>/card | status | tasks/<req>/response
runes/prompts                <- broadcast work
runes/prompts/<req>/progress -> response  <- one shared group, no claim race
runes/tools/<tool>/request | response | error
runes/_log/prompts           <- durable journal
runes/guard/denied           <- refusals (who, what, on what)
$a2a/v1/discovery/<org>/<unit>/<agent_id>
$a2a/v1/tasks/<org>/<unit>/<agent_id>
```

Eleven things that make this genuinely fun to build

1. Deleting the clever part made it stronger. 0.3 removed the entire claim/lease consensus protocol - about 250 lines of "who gets this work?" bookkeeping - and replaced it with MQTT 5 shared subscriptions. The broker now does exactly-once distribution natively. We proved it live: 200 messages, two clients in one shared group, 100/100 each, no dupes, no losses. **The lesson is the fun part: the best distributed-systems code is the code you get to delete.**

2. A hand-rolled MQTT 5 client, and it's only 1 149 lines. The whole adapter - CONNECT/CONNACK property probing, PUBLISH properties, \$share subscriptions, keepalive, reason codes - fits in one readable file with a 33-test codec suite behind it. Writing a wire protocol from the spec and watching two processes talk is one of the purest joys in this trade.

3. The transport is a seam, so the tests never touch a broker. One contract, three implementations: an in-process hub (used by every test), an MQTT 3.1.1 adapter for maximum compatibility, and the MQTT 5 adapter. A test can therefore assert *exactly-once dispatch across two agents* in milliseconds, with no broker, no network, no flake. Swapping the fabric is a constructor argument.

4. A rune is a plugin, and it takes about 30 lines. There is no special-casing:

`Runes::Plugin.names(kind: :rune)` returns the verb list, and dropping a class in adds a verb to the DSL - no engine change:

```
class Runes::Plugins::Greet < Runes::Rune
  plugin :greet, description: "Say hello"
  # Input / Output / Config + execute(input) -> output
end
```

`greet(:hi) { "world" }` now works in every workflow, and inherits the harness's furniture (journal, registry, observatory) because it is a plugin, not a special case. This is the extension story of DSL applied to a workflow DSL, and it is *satisfying*.

5. We adopted another project's DSL, and made the docs unfakeable. Runes runs Roast workflows unmodified - matched in both directions, including the absences (no `respond`, no `on_error`, no `.roast/config`). The discipline that follows is the fun part: `examples/analyze_codebase.rb` *is* the Roast README example, and the compatibility test reads that file, so the docs cannot drift from the artifact. When you use a feature and it turns out your own README lied, you fix both - we did, twice, in one afternoon.

6. Reading a fleet is its own reward. The Rails observatory subscribes to `runes/#` and `$a2a/#` and reconstructs every agent's life: cards, online/offline (including the broker's Last Will), prompt lifecycles, tool calls, A2A handoffs. There is a specific moment of delight in opening a browser and watching a handful of agents pass work to each other on a bus you built.

7. The tests are hermetic, and that's a craft, not a chore. Provider HTTP is faked with a real

`Net::HTTPResponse` over a dup transport; the agent CLI is replaced at a factory seam; the chat backend is injected; the whole workflow suite runs with `command_runner=`, `provider_factory=` and `backend=`.

Consequence: **600 tests, 2 696 assertions, no API key, no network, no broker required** - you can run the entire harness on a plane. *(Round-5 audit found this was false: `test/second_audit_test.rb` dialled the default MQTT port 1883. Fixed - it now spawns its own broker on a free port, and the claim is verified: nothing in the suite reaches the network.)*

8. Ruby is the right language for this, and it's not close. Blocks with `instance_exec`, `method_missing` delegation (so `ruby! (:x).anything` falls through to the value), `define_singleton_method` binding a verb's `X/X!/X?` triple per plugin, `Struct` value objects, `Mutex + Thread` where Python would need a framework. The workflow DSL that results is small enough to read in one sitting - mostly `instance_exec`, `define_singleton_method` and good manners.

9. The honest bits are fun too. `parallel` uses threads instead of the `async` gem - so, yes, a **running** iteration cannot be cancelled, and we write that down. `cmd` inherits Roast's shell semantics for a single command string; that's in the docs **and** in a test. Known gaps are first class in `STATE.md`. Nothing builds trust faster than a project that tells you where it's weak.

10. The fleet got a face. The observatory started as a packet list and is now a console: **workflow runs drawn as timelines** (one bar per step, placed by its offset and scaled to the run, because a 24 ms `cmd` next to a 0.2 ms `ruby` should **look** like that), a **fleet topology** with delegation edges whose width is task volume and colour is failure rate, an **interaction waterfall** where the bars are the **gaps** - so a 30-second planner call looks like 30 seconds instead of hiding in a list of timestamps - and a dashboard with a traffic sparkline, and a **fleet board** that folds the packet stream into Planned / Working / Done and renders it as a Mermaid `kanban` diagram - with `GET /board.mmd` handing the same text to an agent, because a board a program cannot read is a screenshot. All of it from telemetry, none of it guessed. Since then the face learned the two questions a shared bus actually poses: **who really published this** (every packet carries a signature verdict and the signing key's fingerprint, and one `agent_id` under two keys is a finding) and **what was refused** (the guard publishes its denials, so the page draws refusals over time instead of pointing at a log nobody reads).

11. At-least-once is a bug you can make boring. For a while the honesty list below led with *"distribution is exactly-once, execution is at-least-once, and nothing dedupes inbound"* - a sentence that quietly means one prompt can be billed twice. The fix is 187 lines (`lib/runes/request_ledger.rb`): claim the `request_id` before any work is queued, remember what the first copy did, answer a replay from that. Bounded, TTL'd, one TTL (a per-claim window reads as flexibility and behaves as a trap), atomic - sixteen threads, one winner. The satisfying part is not the ledger; it is watching a whole genre of distributed-systems anxiety collapse into a hash with a clock.

Why this is more important than it looks

1. Agents are becoming infrastructure, and infrastructure has requirements. A prompt loop is a demo. The moment an agent can write files, run commands, spend money and talk to other agents, it needs what every other piece of infrastructure needs: **identity, least privilege, audit, and observability**. Runes is built with the guard, the journal, the signatures and the observatory as the product - the model call is the easy part. That inversion is the whole bet, and it's the right one.

2. Non-determinism makes auditability a correctness property. You cannot reproduce an LLM run by re-reading the code. So every prompt lifecycle - request, claim, start, progress, response, plan, tool calls, verifier verdict - is appended to a durable, rotating, `flock`-protected journal and published on the bus, and `bin/runes-replay` shows or replays it. **Your agents' memory should live on your disk, in a format you can read, not in a vendor's dashboard.**

3. "It can do anything" is the threat model, not the feature. A default-deny capability guard decides per tool **and** per action (`fs_read/fs_write/exec`) on the resolved path; tool RPC is off unless explicitly enabled and authenticated with a shared secret compared in constant time; `run_command` rejects path escapes; untrusted tools can run in a real `ruby.wasm` sandbox with a fuel budget; `bin/runes-acl` generates a fail-closed mosquito ACL file so the **broker** enforces it too. Layers, not vibes.

4. On a shared bus, "unsigned" means "unknown", and unknown means "could be anyone". Ed25519 signed envelopes with a per-agent trust store turn `agent_id` from a claim into a fact. Nothing else in the agent ecosystem treats the message bus as an adversarial surface - and a fleet where any process can impersonate any agent is not a fleet, it's a free-for-all.

5. Interop, or you're an island. Two protocols, deliberately chosen: **A2A** so other ecosystems can discover and task your agents, and **MCP** so your tools are consumable by any MCP client (and vice versa). The transport is swappable; the providers are pluggable; the workflow DSL is Roast-compatible. **Adopting Runes doesn't lock you in - and that's the point.** Tools that only work if you never leave are a trap; we'd rather win on merit.

6. Ruby deserves a first-class answer here. The agent boom happened in Python, and the Ruby community has been told, politely, to enjoy its Rails-shaped past. That's nonsense. Ruby is *better* at the two things agent systems actually are: **expressive DSLs** for describing work, and **boring, reliable libraries** for running fleets. Runes is the argument, in code, that Ruby is a serious choice for agent orchestration - not a nostalgic one.

7. Local-first is a political position as much as a technical one. SQLite, mosquito on 127.0.0.1, keys in a `.env` you control, no SaaS in the request path. You can run the whole thing on a laptop on a plane, and you can run it on your own broker with your own ACLs. For anything that touches a company's source code, that is not a nice-to-have.

8. The alternative to a SaaS is not a smaller SaaS - it is a file. Most of what a small team buys a subscription for is an *engine*: turn an idea into a plan, run the plan, verify it, tell someone what to do next, keep the receipts. That engine fits in one workflow you can read (`examples/prospect_pipeline.rb`, 431 lines): it writes a goal, plans a mission kanban, advances every todo through a verifier, computes who to contact next by a rule, drafts the outreach without sending it, and reports. The parts worth paying a SaaS for - identity, tenancy, money, law - stay where they belong, in an app; the part that is actually intelligence becomes malleable, in your repo, runnable by any harness, and it inherits the safety properties a cron job never had. Full write-up, one line tweaks and honest boundaries: `docs/EXAMPLE_CRM_PIPELINE.md`.

Receipts

Because "trust me" is not marketing, here is what's actually in the repo today:

<code>`lib/`</code>	**15 046 lines** across 63 files
<code>`test/`</code>	**11 545 lines** across 47 files
Suite	**618 runs, 2 802 assertions, 0 failures** — no keys, no provider calls, no broker required
Observatory	**215 runs, 1 069 assertions** , **4 860 lines** of Rails 8.1 app code — fleet, runs, topology, traces, who published, and what was refused, all fed through the fleet's own <code>`Runes::Transport`</code>
Executables	**7** : <code>`runes`</code> (TUI), <code>`runes-daemon`</code> , <code>`runes-client`</code> , <code>`runes-mcp`</code> , <code>`runes-replay`</code> , <code>`runes-acl`</code> , <code>`runes-workflow`</code>
Workflow engine	**4 600 lines** (engine, the seven runes, and the rune/cog/plugin support classes), stdlib only — no <code>`async`</code> , no <code>`ruby_llm`</code>
The seven runes	<code>`agent`</code> 728, <code>`chat`</code> 503, <code>`repeat`</code> 202, <code>`cmd`</code> 205, <code>`map`</code> 183, <code>`ruby`</code> 90, <code>`call`</code> 68
MQTT 5 adapter	**1 149 lines** , hand-rolled, live-verified against mosquito 2.1.2 — and it <i>reconnects and re-subscribes</i>
Security surfaces	**1 716 lines** : command policy 309, envelopes 323, identities 240, trust store 184, capability guard 243, guard telemetry 149, credentials 115, RPC auth 99, nonce cache 54
Request ledger	**187 lines** — the execution half of exactly-once (see item 11)
CRM pipeline example	**431 lines** of DSL writing a real pipeline into interoperable files (see item 8)
Dispatcher	**1 656 lines** , down from 1 968 after the fabric/journal/session split

Gem	builds clean — 64 library files, no secrets, no local state, seven binstubs, MIT `LICENSE` shipped
Live proof	**200 messages, one shared group, 100/100 split, no dupes, no losses** (re-verified 2026-09-11 against mosquito 2.1.2), plus a dropped-socket proof that the adapter heals and keeps its subscriptions

Every number above is measured, not remembered - `ruby scripts/receipts.rb` prints all of them (add `--suites` to run the two suites as well). This file is also the source of `docs/WHY_RUNES.pdf`, so a stale number here is a one-command bug.

Run it yourself: `bundle exec rake test`, then `bundle exec ruby demo/smoke.rb` (offline, no key), then `bin/runes-workflow execute examples/analyze_codebase.rb`.

For real work, `examples/prospect_pipeline.rb` (431 lines) is a complete CRM/product pipeline - idea -> goal -> mission kanban -> every todo executed and verified -> next actions per contact -> drafted (never sent) outreach -> weekly report - writing files a human, an agent and a Rails app can all read. See [docs/EXAMPLE_CRM_PIPELINED.md](#).

What we're honest about

A pitch that hides the seams is a pitch you'll resent in a month. So:

- **Distribution is exactly-once; execution is now deduped, in-process.** MQTT 5

shared subscriptions put a prompt in front of exactly one group member, and `Runes::RequestLedger` closes the other half: a redelivered QoS 1 PUBLISH, a session-expiry replay or a publisher retry with the same `request_id` runs the planner **once**, and the replay is answered from the first copy's recorded outcome instead of being silently dropped. Covers prompts, A2A tasks, delegations and direct tool RPCs. The remaining seam is stated, not implied: a request with no `request_id` (a plain prompt) has no identity to dedupe on, the ledger is in-process (a restart forgets it - the MQTT 5 client connects with a clean session, so the broker does not replay either), and a durable ledger is the next step if a deployment turns session expiry on.

- **Workflow runes ask permission only when asked to.** `RUNES_WORKFLOW_POLICY`

puts `cmd`, `agent` and `ruby` behind the capability guard, refusing **before** anything spawns and failing closed on an unreadable policy. It is off by default, because default-deny would break every unmodified Roast file, and its patterns match the command text exactly - so it wants a narrow allowlist, not a wildcard.

- **Token scanning is not a sandbox.** `run_command` refuses interpreters and

paths that leave the workspace, but an explicitly allowlisted binary can still escape. Real confinement needs `sandbox-exec/bwrap`.

- **A2A peer cards are unauthenticated** (discovery-only): the agent id is validated, but a retained card from any publisher is accepted.

- **parallel is threads, not async.** So there is no cooperative cancellation of a running iteration. We traded that for one fewer dependency.

- **MQTT 3.1.1 is compatibility-only** - no shared groups, no properties. It refuses loudly, and it now reconnects like MQTT 5 does.

- **The observatory has no auth**, so read access is a disclosure surface. Fine on localhost; a prerequisite for the "launch workflows from the browser" idea.

- ****The observatory reaches the harness by load path, not by gem**

dependency.** `runes_observer/` puts the sibling checkout's `lib` on the path (`RUNES_HARNESS_LIB`) rather than adding `gem "runes"`, because the harness declares `mqtt` as a runtime dependency and the observer's own transport may not use it. That is a deliberate coupling with a sharp edge: move the app away from the checkout and you must point the variable at a harness.

- **One-shot tool feedback:** a single build plan does not yet loop
plan->execute->feed-back. Missions compensate with verify-and-resume.
Every one of those is a `git grep` away from a doc and a test, which is the point.

Ten minutes in the code

If you read one file, read `lib/runes/request_ledger.rb` (187 lines): a distributed-systems problem met with a hash, a clock and a mutex, plus a comment that admits exactly what it does not cover. After that, in order:

1. `lib/runes/transport.rb` and `lib/runes/transport/mqtt5.rb` - the seam and the hand-rolled protocol behind it. Start at `subscribe` with `group`; that line replaced 250 lines of claim/lease consensus. 2. `lib/runes/core/dispatcher.rb`, `around_handle_prompt` - the inbound choke point: signature gate, then ledger claim, then the work queue. The order of those three *is* the security design. 3. `lib/runes/capabilities/guard.rb` with `lib/runes/guard_telemetry.rb` - what is allowed, and what happens when it is not: a refusal becomes an event on the bus. 4. `runes_observer/app/services/fabric_ingest.rb` - the same seam from the consumer's side, including the retained-replay window and the refusal to ever subscribe as a work-group member. 5. `examples/prospect_pipeline.rb` - what the DSL is *for*: one readable file that turns an idea into a goal, a mission kanban, verified todos, CRM next actions and a report. Read it before deciding a DSL is a toy. 6. `test/request_ledger_test.rb` and `test/request_dedupe_test.rb` - how the ledger is proved: sixteen threads and one winner; a redelivery through a real transport; and two deliberate plain prompts that both run anyway.

```
bundle exec ruby demo/smoke.rb # offline, no key, no broker
bin/runes-workflow execute examples/analyze_codebase.rb
cd runes_observer && bin/rails server -p 3100 # then: bin/runes-ingest
```

Write a verb in 30 lines

The extension story, verbatim from `docs/WORKFLOWS.md` and exercised by `test/plugin_test.rb`:

```
class Runes::Plugins::Greet < Runes::Rune
  plugin :greet, description: "Say hello"

  class Input < Runes::Cog::Input
    attr_accessor :name
    def validate! = raise(InvalidInputError, "'name' is required") if name.nil?
    def coerce(value) = (super; @name = value.to_s)
    end

    class Output < Runes::Cog::Output
      attr_reader :text
      def initialize(text) = (super(); @text = text)
      def raw_text = text
      end

      def execute(input) = Output.new("hello #{input.name}")
    end
  end
end
```

No engine change. `greet(:hi) { "world" }` is now a workflow verb, it shows up in `Runes::Plugin.names(kind: :rune)`, and it will appear in the observatory's plugin catalogue as soon as that lands.

Where this goes next

Four things this document used to promise as future work have shipped, which is the honest version of a roadmap:

- `~~Identity badges and impersonation alerts~~` - done: every packet carries a signature verdict and a key fingerprint, and one `agent_id` under two keys is a finding.
- `~~Guard-decision telemetry~~` - done: `Runes::GuardTelemetry` publishes every refusal on `runes/guard/denied`, and `/security` counts them by tool, agent and action.
- `~~A broker-free mode for the observer~~` - done: ingest goes through `Runes::Transport`, so MQTT 5 properties are visible and `inproc` needs no broker at all.
- `~~A bounded request ledger~~` - done, and it is no longer the first line of the honesty list above.

What is actually left, roughly in the order I would build it:

- **A plugin catalogue** (`/plugins`) generated from `Runes::Plugin.names`, so the "a rune is a plugin" claim is a page rather than a paragraph.
- **Alerts** - impersonation and refusals are already detected; they need a row an operator can acknowledge, not just a panel.
- **The plan chain**: prompt -> plan -> tool calls -> evidence -> verifier verdict on one page, which is the debugging tool for the one-shot-feedback gap.
- **Runs, diffed and replayed** - two runs of the same file side by side, and "re-run just the step that failed".
- **An observatory that is an agent** - its own A2A card and an MCP server, so any agent can ask "what happened on the bus?" mid-task.
- **Retention and rollups at real volume**, and a **flight recorder** that can replay a recorded conversation offline.

The full proposal, with costs and risks, is in [docs/OBSERVATORY_ROADMAP.md](#).

| The paragraph to steal

Runes is a Ruby agent harness that treats the unglamorous half of agent infrastructure as the product: a swappable message fabric (in-process, MQTT 3.1.1, MQTT 5 shared subscriptions), A2A discovery, MCP tools, Ed25519-signed envelopes, a default-deny capability guard, a durable journal with verify-and-resume, and a Rails observatory that watches the whole fleet. Its workflow DSL is Roast-compatible - a Roast file runs unmodified - and every one of its seven verbs is a plugin you can replace in about thirty lines. Nearly fifteen thousand lines of library, eleven thousand lines of hermetic tests, no network required, no vendor in the path.